Section 1: Software and Platform

Software Used

- Python 3.10+
- Jupyter Notebook (for analysis and visualization)

Required Python Packages

The following Python packages must be installed:

- `pandas`
- `numpy`
- `matplotlib`
- `seaborn`
- `scipy`
- `nltk`
- `vaderSentiment`
- `requests`
- `json`

Install all required packages using:

```
pip install pandas numpy matplotlib seaborn scipy nltk vaderSentiment requests
```

If using Anaconda:

```
conda install pandas numpy matplotlib seaborn scipy nltk
pip install vaderSentiment
```

After installing nltk, run the following once in Python to download required resources:

```
import nltk
nltk.download('vader_lexicon')
```

Platform Used

- Operating System: Windows 10 (64-bit)
- Development Environment: Jupyter Notebook

The code should also run on MacOS or Linux provided the same package versions are installed.

Section 2: Instructions for Reproducing Results

Follow these steps carefully to reproduce the results of this study.

Step 1: Download the Repository

Clone the repository:

```
git clone
[https://github.com/abhipas/DS4002_Project1/tree/main](https://github.com/abhipas/DS4002_Project1/tree/main)
```

Or download and extract the ZIP file.

Step 2: Install Required Software

Install Python 3.10+ and all required packages listed in Section 1. Verify installation by running:

```
python --version
```

Step 3: Verify Folder Structure

Ensure that:

- `earnings_events.csv` is located in `data/earnings_data/`
- Raw Reddit JSON files are in `data/raw_reddit_data/`

Note: Do not rename folders or change file paths.

Step 4: Scrape Reddit Data (If Rebuilding Raw Dataset)

Run:

```
python scripts/reddit_scraper.py
```

This will:

- Collect posts and comments from `r/stocks` and `r/wallstreetbets`
- Save JSON files in `data/raw_reddit_data/`

If raw data is already included, skip this step.

Step 5: Data Cleaning and Linking to Earnings

Run:

```
python scripts/preprocessing.py
```

This script:

- Cleans timestamps
- Removes invalid or unmatched observations
- Matches each comment to the most recent prior earnings event
- Computes `hours_after_earnings`
- Saves merged dataset to `data/processed_data/merged_dataset.csv`

Step 6: Sentiment Scoring

Run:

```
python scripts/sentiment_scoring.py
```

This script:

- Applies VADER sentiment analysis
- Generates:
- `vader_neg`
- `vader_neu`

- `vader_pos`
- `vader_compound`
- `sentiment_label`
- Saves updated dataset

Step 7: Apply Post-Earnings Reaction Window

The preprocessing script filters comments to the 0–168 hour (7-day) window after earnings. Verify that:

- `hours_after_earnings` >= 0
- `hours_after_earnings` <= 168

Step 8: Run Statistical Analysis

Run:

```
python scripts/statistical_tests.py
```

This performs:

- Welch two-sample t-test comparing mean VADER compound sentiment
- Comparison of proportion of negative comments
- Comparison of engagement volume
- Timing distribution analysis

Significance level:

- (two-sided test)

Results are saved to: `results/tables/statistical_results.csv`

Step 9: Generate Visualizations

Run:

```
jupyter notebook notebooks/03_statistical_analysis.ipynb
```

Execute all cells to generate:

- Sentiment comparison plots
- Engagement comparison plots
- Timing distribution plots

Figures will be saved.

Step 10: Verify Results

To confirm successful reproduction:

- Mean sentiment values should match the statistical output file.
- P-values should align with those reported in the project report.
- Figures should match those shown in the output section.

If all steps were completed correctly, your results should replicate the reported conclusions regarding:

- Differences in average sentiment between earnings beats and misses
- Differences in engagement volume
- Timing patterns of investor discussion